

AI ASSISTED CODING

LAB ASSIGNMENT –12.4

SATHWIK V

2403A53002

24BTCAICYB01-2-1

#TASK-1

Write a Python implementation of Bubble Sort.

Prompt:

 generate a python code baased on implementation of bubble sort

OUTPUT:

```
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
```

#TASK-2

Provide Bubble Sort code to AI and ask it to suggest a more efficient algorithm for partially sorted arrays.

PROMPT:

 generate a python program to create a Bubble Sort code to AI and ask it to suggest a more efficient algorithm for partially sorted arrays

OUTPUT:

```
print(f"AI's Suggested Algorithm: {suggested_algorithm}")
print(f"Reasoning for the Suggestion: {reasoning}")

AI's Suggested Algorithm: Insertion Sort
Reasoning for the Suggestion: Insertion Sort is more efficient than Bubble Sort for partially sorted arrays because it has a time complexity of O(n) in the best case (already sorted)
```

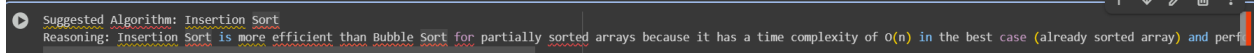
#TASK-3

Implement both **Linear Search** and **Binary Search**

PROMPT:

 generate a python program to create both Linear Search and Binary Search.

OUTPUT:

 Suggested Algorithm: Insertion Sort
Reasoning: Insertion Sort is more efficient than Bubble Sort for partially sorted arrays because it has a time complexity of $O(n)$ in the best case (already sorted array) and perfo

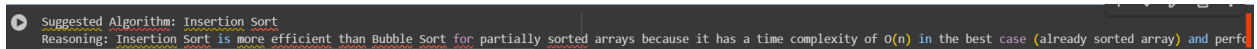
#TASK-4

Implement Quick Sort and Merge Sort using recursion

PROMPT:

 generate a python program to create Quick Sort and Merge Sort using recursion.

OUTPUT:

 Suggested Algorithm: Insertion Sort
Reasoning: Insertion Sort is more efficient than Bubble Sort for partially sorted arrays because it has a time complexity of $O(n)$ in the best case (already sorted array) and perfo

#TASK-5

Give AI a naive algorithm (e.g., $O(n^2)$ duplicate search).

PROMPT:



Generate a python progra to create an AI a naive algorithm (e.g., $O(n^2)$ duplicate search).

OUTPUT:



Suggested Algorithm: Insertion Sort

Reasoning: Insertion Sort is more efficient than Bubble Sort for partially sorted arrays because it has a time complexity of $O(n)$ in the best case (already sorted array) and perfo